

To Infinity and Beyond: a GPU-driven memory sharing pipeline to generate and process infinite synthetic data

Daniele Della Pietra

University of Trento
Trento, Italy

daniele.dellapietra@studenti.unitn.it

Gino Lanzo Hahn

University of Trento
Trento, Italy

gino.lanzohahn@studenti.unitn.it

Nicola Garau

University of Trento
Trento, Italy

nicola.garau@unitn.it

Abstract

What if data generation, manipulation, and training could all happen entirely on the GPU, without ever touching the RAM or the CPU? In this work, we present a novel pipeline based on Unreal Engine 5, which allows us to generate, render, and process graphics data entirely on the GPU. By keeping the data stored in GPU memory throughout all the steps, we bypass the traditional bottlenecks related to CPU-GPU transfers, significantly accelerating data manipulation and enabling fast training of deep learning algorithms. Traditional storage systems impose latency and capacity limitations, which become increasingly problematic as data volume increases. Our method demonstrates substantial performance improvements on multiple benchmarks, offering a new paradigm for integrating game engines with data-driven applications. More information on our project page: <https://mmlab-cv.github.io/Infinity/>

CCS Concepts

• **Computing methodologies** → **Computer graphics**; *Artificial intelligence*; *Computer vision*; *Machine learning*; *Graphics processors*.

Keywords

Computer Graphics, GPU, Datasets, AI, Deep Learning

ACM Reference Format:

Daniele Della Pietra, Gino Lanzo Hahn, and Nicola Garau. 2025. To Infinity and Beyond: a GPU-driven memory sharing pipeline to generate and process infinite synthetic data. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters (SIGGRAPH Posters '25)*, August 10–14, 2025, Vancouver, BC, Canada. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3721250.3742983>

1 Introduction and Motivation

During the past few years, game engines have proven to be more than capable at generating and rendering high resolution, photorealistic scenes, with extensive customization options. For this reason, they are the ideal tools for generating synthetic datasets [Pollok et al. 2019; Shah et al. 2017] that can be used to speed up data-driven applications. However, the typical workflow—(1) build 3D scenes in a game engine, (2) render and write its data to disk, and

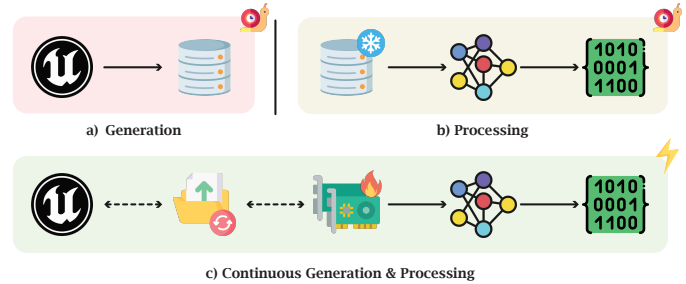


Figure 1: Overview of a typical synthetic data generation pipeline (a) and data processing via an algorithm or neural network (b), versus our fast unified synthetic data generation and processing pipeline (c).

finally (3) use that static data as input to an algorithm or neural network—suffers from repeated CPU–GPU round-trips and I/O stalls. The main bottleneck of this approach is usually the storage, which can contain a finite amount of static data, with a reduced access speed from the graphics hardware. Our contribution is twofold: (i) generating an infinite amount of data in real-time on the GPU, and (ii) consuming it without storing it anywhere, bypassing the delays due to storage access. Fig. 1 shows a description of our pipeline.

2 Method

We design two solutions to enable the sharing process in a synchronized way, without relying on the physical storage at any point. We evaluate on three tasks—mesh transfer, texture rendering, and training throughput—comparing a standard DataLoader (CPU-disk), shared-RAM, and fully GPU-resident pipeline over 1000 runs.

RAM Sharing: On the engine side, we develop a simple plugin that allocates a shared memory portion that lives in the RAM via a dedicated C++ library (UE::Learning::SharedMemory). On the Python side, we develop a library enabling a communication channel with UE5, that can read data synchronously from it, taking care of concurrent accesses via a flag system.

GPU Sharing: We leverage CUDA IPC for zero-copy GPU data exchange: at startup a device buffer is allocated and its memory and event handles registered under a GUID. The engine exports a Direct3D handle that CUDA imports alongside an IPC event. Each frame, these handles are opened to enqueue a GPU-only copy and signal completion; consumers then map the buffer and wait on the event. This works for meshes, textures, or tensors, enabling seamless on-the-fly GPU sharing with no CPU overhead.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SIGGRAPH Posters '25, Vancouver, BC, Canada
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1549-5/25/08
<https://doi.org/10.1145/3721250.3742983>

Table 1: Transferring 3D meshes from UE5 to Python and rendering them with PyOpenGL. Despite obtaining the best results when fully utilizing the GPU, we show how strategically allocating a shared portion of the RAM can also lead to good performances when compared to loading data from local storage. However, our approach on the GPU shows remarkably constant performances across all the models, showcasing impressive scalability potential.

Metric	RAM	GPU	RAM	GPU	RAM	GPU	RAM	GPU	RAM	GPU	RAM	GPU	RAM	GPU	RAM	GPU
Model	Suzanne		Spot		Teapot		Bunny		Nefertiti		Lucy		Armadillo		Dragon	
Vertices	507		2930		3644		35947		49971		49987		49990		125066	
Faces	968		5856		6320		69451		99938		99970		99976		249882	
Data copy (ms)	0.133	0.293	0.141	0.311	0.154	0.295	0.390	0.301	1.013	0.308	1.046	0.307	1.007	0.306	2.218	0.301
Rendering (ms)	0.175	0.197	0.179	0.203	0.182	0.195	0.288	0.197	0.600	0.202	0.595	0.204	0.570	0.201	1.231	0.198
Frame time (ms)	0.330	0.540	0.343	0.563	0.358	0.537	0.699	0.547	1.635	0.557	1.664	0.558	1.599	0.555	3.473	0.548
Avg. FPS	3028	1851	2915	1778	2794	1862	1430	1829	612	1795	601	1791	625	1803	288	1826

Table 2: Transferring RGBA rendered images at various resolutions from UE5 to Python. After the data copy, the images are rendered once again using OpenCV. In order to do that, the images need to be copied back to CPU memory, thus the increase in the Rendering row.

Metric	RAM	GPU	RAM	GPU	RAM	GPU	RAM	GPU	RAM	GPU
Resolution	128	128	256	256	512	512	1024	1024	2048	2048
Data copy (ms)	0.035	0.362	0.259	0.410	0.770	0.429	2.809	0.491	13.144	0.427
Rendering (ms)	10.840	10.883	10.703	10.835	11.047	10.817	15.194	10.889	43.819	23.588
Total time (ms)	10.882	11.249	10.969	11.249	11.826	11.250	18.012	11.783	56.976	24.019
Avg. FPS	92	89	91	89	85	89	56	85	18	42

Table 3: Training on a synthetic, real-time-generated dataset of renderings at various resolutions, versus training on the same subset of pre-rendered images in Pytorch. Frame time is the time spent grabbing a single batch (including UE5’s rendering time for our GPU scenario) and storing it in a Pytorch tensor. ResNet-50 describes the overall iterations per second, including the network processing time.

Metric	DataLoader	GPU	DataLoader	GPU	DataLoader	GPU	DataLoader	GPU	DataLoader	GPU
Resolution	128	128	256	256	512	512	1024	1024	2048	2048
Frame time	0.4 ms	3 ms	1 ms	3 ms	5 ms	3 ms	14 ms	3 ms	50 ms	3 ms
Resnet-50	79.16	57.61	76.63	63.34	61.71	61.60	24.53	31.06	5.80	7.89

3 Experiments and Results

We run all of our experiments on a single NVIDIA RTX 4090.

Mesh transfer: We first measure the cost of transferring 3D meshes from Unreal Engine 5 into Python and rendering them via Py-OpenGL. Table 1 reports data-copy time (UE5 to Python, with no additional processing), rendering time (in Python, using GL), overall frame time, and average FPS on eight common meshes taken from the Stanford 3D Scanning Repository. Our results in Table 1 demonstrate that an end-to-end GPU approach offers dramatic speedups (up to 12×) and constant performance as scene complexity grows.

Render target: We render RGBA textures at increasing resolutions (128 px–2048 px) in UE5 and share them with a Python process over either RAM or GPU. The shared images are then visualized using OpenCV. The results can be seen in Table 2. Our GPU-resident pipeline removes the resolution-dependent bottleneck in both data transfer and image-processing stages. The GPU copy remains near 0.4–0.5 ms even at 2048 px, while the RAM variant shows a far bigger variance (13 ms for a 2048 px image).

Training on a dataset: An application of our method is generating on-the-fly, high-quality infinite data for real-time neural network training. Our method can bypass the reliance on storage and provide a constant stream of data during training, effectively acting like an infinite batch generator. We compare our GPU pipeline that loads images into PyTorch tensors with a standard Pytorch DataLoader and train a ResNet50 network on the fly. Table 3 shows that the frame time for our GPU pipeline remains almost constant at different resolutions, differently from the standard DataLoader.

Acknowledgments

Funded by the European Union - Next Generation EU, Mission 4 Component 2 - CUP E63C22000970007.

References

- Thomas Pollok, Lorenz Junglas, Boitumelo Ruf, and Arne Schumann. 2019. UnrealGT: Using Unreal Engine to Generate Ground Truth Datasets. In *Advances in Visual Computing : 14th International Symposium on Visual Computing, ISVC 2019, Lake Tahoe, NV, USA, October 7–9, 2019, Proceedings, Part I*. Ed.: George Bebis. Springer, 670–682. doi:10.1007/978-3-030-33720-9_52
- Shital Shah, Debadepta Dey, Chris Lovett, and Ashish Kapoor. 2017. AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles. In *Field and Service Robotics*. arXiv:arXiv:1705.05065 <https://arxiv.org/abs/1705.05065>